

Manuale per l'uso di Docker

NECKER

16 Dicembre 2025

Indice

1	Architettura e Concetti Fondamentali	2
1.1	L'Immagine (The Class)	2
1.2	Il Container (The Object)	2
2	Deep Dive: Docker Internals e Kernel Linux	3
2.1	Namespaces: L'Illusione dell'Isolamento	3
2.2	Cgroups (Control Groups)	3
2.3	Union File System & Copy-on-Write (CoW)	4
2.3.1	Il Meccanismo Copy-on-Write (CoW)	4
3	Strategie di Build Avanzate	5
3.1	Ottimizzazione della Cache dei Layer	5
3.2	Exec Form vs Shell Form (Gestione dei Segnali)	6
3.3	CMD vs ENTRYPOINT: La Matrice di Interazione	6
3.4	Multi-Stage Builds: Separare Build e Runtime	7
3.5	Il file .dockerignore	7
4	Networking: Come Comunicano i Container	8
4.1	Modalità di Rete a Confronto	8
4.2	Comunicazione Esterna: Il Port Mapping (DNAT)	9
4.3	Comunicazione Interna: DNS e Service Discovery	9
5	Persistenza e Storage Strategy	10
5.1	Volumes (Produzione / Database)	10
5.2	Bind Mounts (Sviluppo / Configurazione)	10
5.3	tmpfs Mounts (Sicurezza)	10
6	Docker Compose: Orchestrazione e Infrastructure as Code	11
6.1	Il Cambio di Paradigma: Imperativo vs Dichiarativo	11
6.2	Anatomia di un docker-compose.yml	12
6.3	Analisi delle Keywords	12
6.4	Il Workflow Operativo	13
7	Comandi Cheat-Sheet	13

1 Architettura e Concetti Fondamentali

Virtualizzazione: VM vs Container

La differenza sostanziale risiede nel *livello di astrazione*.

- **Virtual Machines (Hardware Virtualization):** Una VM emula un'intera macchina fisica. Sopra l'Hypervisor gira un **Guest OS** completo (con il suo Kernel, driver, gestione memoria).
 - *Pro:* Isolamento totale.
 - *Contro:* Pesante (GB di spazio), avvio lento (boot dell'OS), spreco di risorse (duplicazione del Kernel).
- **Containers (OS Virtualization):** Un container virtualizza il Sistema Operativo, non l'hardware. Tutti i container condividono lo stesso **Host Kernel**. Docker Engine isola i processi (usando Namespaces) e le risorse (usando Cgroups), ma non emula hardware.
 - *Pro:* Leggerissimo (MB), avvio istantaneo (è solo un processo), densità elevata.
 - *Contro:* Limitato all'architettura del sistema operativo.

Il Paradigma OOP: Immagini e Container

Per uno sviluppatore, il modo migliore per comprendere Docker è attraverso l'analogia con la Programmazione a Oggetti (Classi e Oggetti).

1.1 L'Immagine (The Class)

In programmazione, una **Classe** è un modello statico, un "blueprint" che definisce proprietà e metodi, ma non "vive" in memoria di per sé fino a quando non viene istanziata.

Nel mondo Docker, l'**Immagine** è l'equivalente della Classe (o meglio, della classe compilata/binaria):

- **Immutabile:** Una volta costruita ('docker build'), non può essere modificata. Proprio come non modifichi il bytecode di una classe a runtime.
- **Statica:** È un pacchetto di sola lettura che contiene tutto il necessario per l'esecuzione: codice sorgente, runtime (es. JVM, Node), librerie di sistema e dipendenze.
- **Layered:** È costruita a strati. Ogni istruzione nel Dockerfile crea un nuovo layer immutabile.

1.2 Il Container (The Object)

In programmazione, un **Oggetto** è un'istanza di una classe. Occupa memoria, ha uno stato e un ciclo di vita. Puoi creare infiniti oggetti dalla stessa classe.

Nel mondo Docker, il **Container** è l'equivalente dell'Oggetto:

- **Runtime:** È l'immagine che prende vita. Quando lanci 'docker run', stai facendo un'operazione di *instantiation* ('new Image()').
- **Mutable (con effimerità):** A differenza dell'immagine, il container può essere modificato. Docker aggiunge un sottile *Writable Layer* sopra i layer dell'immagine. Se il container scrive un file, lo scrive qui.
- **Isolato:** Se crei due container dalla stessa immagine (es. due database Postgres), essi sono due istanze separate. Se modifichi i dati in uno, l'altro non viene influenzato (proprio come modificare 'obj1.x' non cambia 'obj2.x').

Dockerfile $\approx$ Codice Sorgente (.java / .cpp)

Docker Image $\approx$ Classe Compilata / Eseguitibile statico

Docker Container $\approx$ Oggetto in Heap / Processo in esecuzione

2 Deep Dive: Docker Internals e Kernel Linux

Docker non è una tecnologia monolitica, ma un wrapper (precedentemente tramite LXC, ora tramite `libcontainer/runc`) che orchestra tre primitive fondamentali del Kernel Linux. Capire questi concetti è la chiave per il debugging avanzato e l'ottimizzazione delle performance.

2.1 Namespaces: L'Illusione dell'Isolamento

I Namespaces sono una feature del kernel che partiziona le risorse del sistema in modo che un set di processi veda solo una parte delle risorse. Quando Docker avvia un container, invoca la syscall `clone()` passando specifici flag (es. `CLONE_NEWPID`, `CLONE_NEWNET`).

- **PID Namespace (Process ID):** Isola l'albero dei processi.
 - *Inside:* Il processo principale del container vede se stesso come **PID 1**. Questo è cruciale perché il PID 1 ha responsabilità speciali (gestione segnali, raccolta processi orfani/zombie).
 - *Outside:* Sull'host, quel processo ha un PID normale (es. 4532).
- **NET Namespace (Networking):** Fornisce al container uno stack di rete completo e privato: proprie interfacce (es. `eth0`), tabelle di routing, regole iptables e socket. Docker usa *veth pairs* (cavi ethernet virtuali) per collegare il namespace del container al bridge dell'host (`docker0`).
- **MNT Namespace (Mount Points):** Permette al container di avere una visione diversa del filesystem. Un processo può montare/smontare directory senza influenzare l'host. È ciò che permette al container di vedere la sua root / diversa da quella dell'host.
- **USER Namespace (Sicurezza Avanzata):** Permette il mapping degli ID utente. Il processo può essere `root` (UID 0) all'interno del container, ma essere mappato a un utente senza privilegi (es. UID 1000) sull'host. Questo mitiga drasticamente i rischi di *Container Breakout*.

2.2 Cgroups (Control Groups)

Mentre i Namespaces gestiscono cosa il container può *vedere*, i Cgroups gestiscono cosa il container può *usare*. Sono gerarchie di directory in `/sys/fs/cgroup`.

- **Resource Limiting:** Puoi impostare un hard limit sulla memoria (es. 512MB). Se il container supera il limite, il Kernel invoca l'**OOM Killer** (Out of Memory Killer) e termina il processo.
- **Prioritization:** Puoi assegnare pesi alla CPU (CPU Shares). Se il sistema è scarico, un container può usare il 100% della CPU. Se c'è contesa, i Cgroups garantiscono le risorse in base ai pesi assegnati.
- **Accounting:** Cgroups traccia quanto consuma ogni container. È la base su cui funzionano comandi come `docker stats`.

2.3 Union File System & Copy-on-Write (CoW)

Questa è la magia che rende i container rapidi da avviare e leggeri su disco. Docker usa driver di storage (solitamente **Overlay2**) che implementano un Union File System.

Immagina l'immagine come una pila di fogli trasparenti (Layers).

1. **LowerDir (Image Layers):** Sono i layer dell'immagine scaricata (es. Ubuntu base + Python + App Code). Sono **Read-Only**. Non possono mai essere modificati.
2. **UpperDir (Container Layer):** Quando avvii un container, Docker aggiunge un layer vuoto in cima. È l'unico strato **Read-Write**.
3. **Merged View:** Il container vede l'unione di questi strati come un unico filesystem coerente.

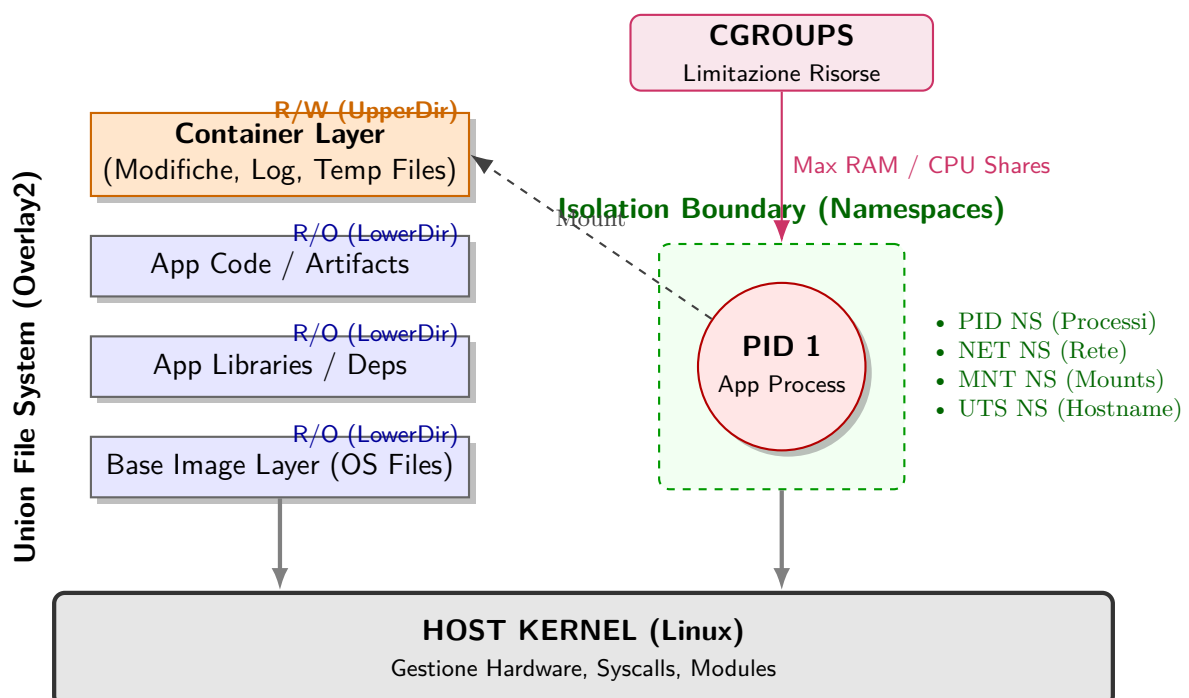
2.3.1 Il Meccanismo Copy-on-Write (CoW)

Cosa succede quando il container vuole modificare un file che esiste in un layer dell'immagine (Read-Only)?

- **Read:** Se il container legge un file, lo legge direttamente dal layer inferiore (velocità nativa).
- **Write:** Se il container vuole modificare `/etc/config.ini` che si trova nell'immagine, il driver OverlayFS prima **copia** il file dal layer inferiore (LowerDir) al layer scrivibile superiore (UpperDir). La modifica avviene sulla copia.
- **Delete:** Se "cancelli" un file dell'immagine, Docker crea in realtà un *Whiteout File* nel layer superiore che nasconde il file sottostante senza rimuoverlo davvero.

Performance Tip per Programmatori

La strategia CoW ha un costo di I/O. **Best Practice:** Non usare il filesystem del container per carichi di scrittura intensivi (es. Database) dato che all'eliminazione del container verranno eliminati. Usa i **Volumi** (directory del computer collegate al container), che bypassano il driver OverlayFS e scrivono direttamente sul disco dell'host a velocità nativa.



3 Strategie di Build Avanzate

Il Dockerfile non è solo uno script di installazione; è la definizione immutabile dell'ambiente di produzione. Scriverlo male significa avere build lente, immagini giganti e vulnerabilità di sicurezza.

3.1 Ottimizzazione della Cache dei Layer

Ogni istruzione nel Dockerfile ('RUN', 'COPY', 'ADD') genera un nuovo layer. Docker utilizza un sistema di caching aggressivo: se il contenuto di un layer non è cambiato rispetto alla build precedente, Docker riutilizza il layer cachato.

Il concetto chiave è l'ordine delle istruzioni: bisogna posizionare le istruzioni che cambiano raramente (es. installazione dipendenze di sistema) in alto, e quelle che cambiano spesso (es. codice sorgente) in basso.

```
1  # Definisce l'immagine di partenza (OS Linux Debian-based + Interprete Python
2  3.9).
3  FROM python:3.9-slim
4
5  # Imposta variabili d'ambiente globali per il container:
6  # - PYTHONDONTWRITEBYTECODE: Evita la creazione di file .pyc (inutili in
7  container stateless).
8  # - PYTHONUNBUFFERED: Invia i log direttamente a stdout senza buffering (
9  essenziale per Docker).
10 ENV PYTHONDONTWRITEBYTECODE=1 \
11     PYTHONUNBUFFERED=1
12
13 # Crea la directory /app (se non esiste) e imposta il puntatore di lavoro lì (
14 equivalente a mkdir + cd).
15 WORKDIR /app
16
17 # Copia il file dei requisiti dall'host al filesystem del container.
18 # Eseguito separatamente per sfruttare la Cache dei Layer se le dipendenze non
19 cambiano.
20 COPY requirements.txt .
21
22 # Esegue l'installazione delle librerie all'interno del container durante la
23 build.
24 # --no-cache-dir evita di salvare la cache di pip, riducendo le dimensioni dell'
25 immagine.
26 RUN pip install --no-cache-dir -r requirements.txt
27
28 # Copia tutto il resto del codice sorgente dalla cartella corrente dell'host alla
29 WORKDIR del container.
30 COPY . .
31
32 # Crea un nuovo utente di sistema chiamato 'myuser' per evitare di eseguire l'app
33 come Root.
34 RUN useradd -m myuser
35
36 # Effettua lo switch dell'utente attivo. Tutte le istruzioni successive (e l'app)
37 gireranno come 'myuser'.
38 USER myuser
39
40 # Dichiaro (a scopo documentale) che il servizio ascolta sulla porta 5000 TCP.
41 EXPOSE 5000
42
43 # Definisce il comando che viene lanciato quando il container si avvia (PID 1).
44 # Usiamo la sintassi a array JSON (Exec form) per gestire correttamente i segnali
45 di stop.
46 CMD ["python", "app.py"]
```

3.2 Exec Form vs Shell Form (Gestione dei Segnali)

Questo è un dettaglio che il 90% dei tutorial ignora, ma è vitale per la stabilità in produzione. Esistono due modi per scrivere 'CMD' o 'ENTRYPOINT'.

- **Shell Form:** `CMD python app.py`
 - Docker avvia il comando avvolgendolo in `/bin/sh -c`.
 - **Problema:** La shell diventa il processo PID 1, e la tua app diventa un processo figlio. Le shell spesso **non inoltrano i segnali** (come `SIGTERM` o `SIGINT`) ai figli.
 - *Risultato:* Quando fai `docker stop`, il container non si ferma dolcemente, ma aspetta 10 secondi e viene ucciso brutalmente (`SIGKILL`), corrompendo potenzialmente i dati.
- **Exec Form (JSON Array):** `CMD ["python", "app.py"]`
 - Docker avvia l'eseguibile direttamente.
 - La tua app è **PID 1**. Riceve immediatamente i segnali di stop e può eseguire lo shutdown pulito (chiudere connessioni DB, finire richieste in corso).
 - **Best Practice:** Usa SEMPRE la notazione a parentesi quadre.

3.3 CMD vs ENTRYPOINT: La Matrice di Interazione

Spesso confusi, lavorano insieme.

- **ENTRYPOINT:** Definisce l'eseguibile "fisso" del container.
- **CMD:** Definisce gli argomenti di default passati all'ENTRYPOINT.

Pattern Comune: L'Eseguibile Configurabile

Se vuoi un container che si comporti come un comando binario:

```
1     ENTRYPOINT ["/bin/ping"]
2     CMD ["localhost"]
3
```

- `docker run myimg` → Esegue `ping localhost`
- `docker run myimg google.com` → Esegue `ping google.com` (CMD sovrascritto)

3.4 Multi-Stage Builds: Separare Build e Runtime

Introdotta per risolvere il problema delle immagini enormi nei linguaggi compilati (Go, Java, C++, Rust) o nei frontend moderni (React/Angular).

Il problema storico era: "Ho bisogno di `gcc`, `maven` o `node_modules` per compilare, ma non li voglio in produzione perché pesano e sono insicuri".

Con il Multi-Stage, usi un'immagine "grassa" per compilare e copi solo l'artefatto finale in un'immagine "magra".

```
1  # --- STAGE 1: Il Cantiere (Builder) ---
2  # Usiamo un'immagine completa con compilatore Go
3  FROM golang:1.18 AS builder
4  WORKDIR /app
5  COPY . .
6  # Compiliamo un binario statico (senza dipendenze dinamiche)
7  RUN CGO_ENABLED=0 GOOS=linux go build -o myapp .
8
9  # --- STAGE 2: Il Prodotto Finito (Runtime) ---
10 # Usiamo Alpine (5MB) o addirittura Scratch (0MB - immagine vuota)
11 FROM alpine:latest
12 WORKDIR /root/
13 # Copiamo SOLO il file binario dallo stage precedente
14 COPY --from=builder /app/myapp .
15
16 # Risultato: Un'immagine di 10MB invece di 800MB
17 CMD ["/myapp"]
18
```

Listing 1: Esempio Go Multi-Stage

3.5 Il file `.dockerignore`

Spesso dimenticato, funziona come `.gitignore`. Prima di iniziare la build, il client Docker invia tutto il contenuto della cartella corrente (Build Context) al demone Docker.

Se non hai un `.dockerignore`, potresti inviare inavvertitamente:

- La cartella `.git` (enorme e inutile).
- Cartelle locali come `venv` o `node_modules` (rompono la build Linux se sviluppi su Mac/Windows).
- File con segreti (`.env`).

Esempio `.dockerignore`:

```
.git
.env
venv
__pycache__
node_modules
Dockerfile
.dockerignore
```

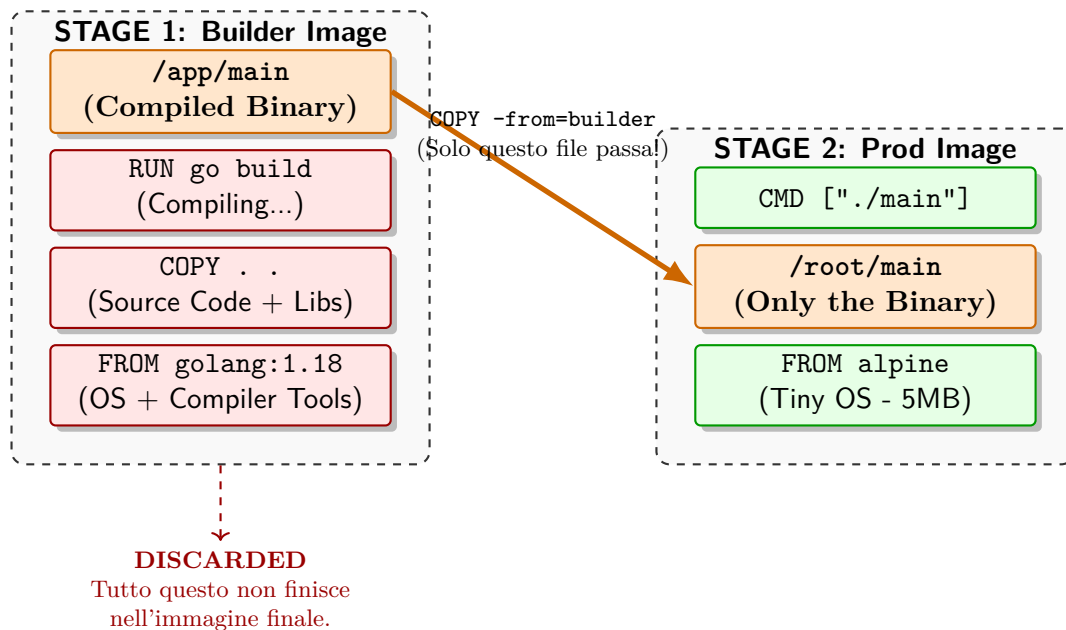


Figura 1: Visualizzazione del Multi-Stage Build. Notare come l'intero ambiente di compilazione (Stage 1) venga scartato, trasferendo solo l'eseguibile finale nell'ambiente di produzione (Stage 2).

4 Networking: Come Comunicano i Container

Docker usa primitive di rete Linux standard per indirizzare il traffico dei vari pacchetti.

1. L'Hardware Virtuale: Bridge e VETH Pairs

Quando Docker parte, crea uno switch virtuale software sull'host chiamato `docker0`.

Ma come si collega il container a questo switch?

Tramite le **VETH Pairs (Virtual Ethernet)**.

Immagina un cavo Ethernet con due estremità:

- Lato Container (`eth0`):** Risiede dentro il Network Namespace del container. Per il processo, questa è una scheda di rete fisica reale.
- Lato Host (`vethXYZ`):** L'altra estremità del cavo rimane nel Namespace dell'Host ed è collegata ("pluggata") al bridge `docker0`.

Risultato: Tutti i container collegati al bridge possono scambiarsi pacchetti come se fossero computer collegati allo stesso switch fisico.

4.1 Modalità di Rete a Confronto

A. BRIDGE (Default)

Il container è isolato. Ha il suo IP privato (es. `172.17.0.2`) e non è raggiungibile dall'esterno.

- Pro:** Isolamento completo, nessuna collisione di porte.
- Contro:** Richiede NAT per uscire e Port Mapping per entrare.

B. HOST (`-network host`)

Rimuove l'isolamento di rete. Il container "prende in prestito" la scheda di rete dell'host.

- Pro:** Prestazioni native (zero overhead NAT). Ideale per VoIP o streaming.
- Contro:** Se il container ascolta sulla porta 80, occupa la porta 80 reale della macchina. Non puoi avviare due container uguali.

4.2 Comunicazione Esterna: Il Port Mapping (DNAT)

Come funziona realmente `-p 8080:80`? Docker non agisce come un proxy user-space, ma manipola il Kernel tramite **IPTables**.

Docker inserisce una regola nella catena **PREROUTING** del firewall Linux:

```
1 # Pseudo-regola IPTables creata da Docker
2 -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:80
3
```

Traduzione: "Qualsiasi pacchetto TCP che arriva alla porta 8080 dell'Host, cambia l'indirizzo di destinazione nell'IP del Container alla porta 80". Questo è **Destination NAT**.

4.3 Comunicazione Interna: DNS e Service Discovery

Un errore comune è usare gli IP dei container (che cambiano al riavvio). Docker risolve questo problema con un server DNS interno.

- **Il Server DNS (127.0.0.11):** Docker inietta questo nameserver nel file `/etc/resolv.conf` di ogni container.
- **Service Discovery:** In Docker Compose, viene creata una rete custom. Il DNS associa automaticamente il **nome del servizio** (es. `db`, `backend`) al suo IP corrente.

Attenzione a Localhost!

Dentro un container, `localhost` (127.0.0.1) è il container stesso! Se il tuo backend Node.js cerca di connettersi a Mongo usando `localhost:27017`, fallirà perché cerca Mongo *dentro* il container Node (ne stesso ma nel posto sbagliato). **Soluzione:** Usa il nome del servizio (es. `mongodb:27017`).

5 Persistenza e Storage Strategy

Il filesystem del container è progettato per essere effimero. Capire *dove* vanno i dati è la differenza tra un'app robusta e una perdita di dati catastrofica.

5.1 Volumes (Produzione / Database)

I volumi sono oggetti di prima classe gestiti da Docker. Risiedono in una parte del filesystem dell'host gestita ESCLUSIVAMENTE da Docker (su Linux: `/var/lib/docker/volumes/`).

- **Vantaggi:**
 - Indipendenti dalla struttura delle cartelle dell'host.
 - Prestazioni I/O native (specialmente su Docker Desktop per Mac/Windows, dove i bind mount sono lenti).
 - Facili da backuppare e migrare (`docker volume create`, `docker volume ls`).
- **Best Use Case:** Database (Postgres, MySQL), storage di chiavi persistenti, log di produzione.

5.2 Bind Mounts (Sviluppo / Configurazione)

Mappano un file o una cartella esatta dall'host dentro il container. `-v /home/user/project:/app`

- **Vantaggi:**
 - **Hot Reloading:** Modifichi il codice sorgente nel tuo editor sull'host, e il container vede la modifica immediatamente. Essenziale per lo sviluppo web (Node, Python, PHP).
 - Condivisione file di configurazione veloci (es. passare un `nginx.conf` al volo).
- **Il Problema dei Permessi (Permission Hell):** Se il processo nel container gira come `root` (default) e crea un file in un bind mount, quel file apparirà sull'host come proprietà di `root`. Tu (utente normale) non potrai cancellarlo o modificarlo senza `sudo`. *Soluzione:* Usare il flag `user:${UID}:${GID}` nel compose file per allineare gli utenti.

5.3 tmpfs Mounts (Sicurezza)

I dati vengono scritti solo nella RAM dell'host. Mai su disco.

- **Use Case:** Memorizzare segreti, token temporanei o dati che non devono assolutamente persistere se il container si spegne.

Storage Decision Matrix

- Codice Sorgente (Dev) → **Bind Mount**
- Database / Dati Persistenti → **Volume**
- File Configurazione Statici → **Bind Mount** o **ConfigMap** (in K8s)
- Token / Segreti → **tmpfs**

6 Docker Compose: Orchestrazione e Infrastructure as Code

Mentre `docker run` è ottimo per avviare container "usa e getta", è inadatto per gestire architetture complesse. **Docker Compose** è lo strumento che permette di definire e gestire applicazioni multi-container.

6.1 Il Cambio di Paradigma: Imperativo vs Dichiarativo

- **Docker Run (Imperativo):** Devi dire al sistema *cosa fare* passo dopo passo. *"Scarica questo, poi avvia quello, poi collega la rete..."*. È manuale, pronò a errori e difficile da condividere con il team.
- **Docker Compose (Dichiarativo):** Descrivi in un file (YAML) *come deve essere* lo stato finale del sistema. *"Voglio un servizio web connesso a un database postgres"*. Docker si occupa di creare le risorse necessarie per raggiungere quello stato.

I Vantaggi rispetto a Docker Run

1. **Infrastructure as Code (IaC):** L'intera infrastruttura è definita in un file (`docker-compose.yml`) che può essere versionato con Git. Un nuovo sviluppatore deve solo fare `git clone` e `docker compose up` per avere l'ambiente pronto.
2. **Networking Automatico:** Compose crea automaticamente una rete interna dedicata al progetto. I servizi si "vedono" usando il loro nome (Service Discovery), senza dover gestire IP.
3. **Persistenza della Configurazione:** Non devi ricordare stringhe chilometriche di flag `-v`, `-p`, `-e`. È tutto scritto nel file.

6.2 Anatomia di un docker-compose.yml

Il file YAML è sensibile all'indentazione. Ecco un esempio realistico di uno stack Full-Stack (Python + Postgres).

```
1  version: '3.8' # Versione della specifica
2
3  services:      # Definizione dei container (Servizi)
4
5  # --- SERVIZIO 1: IL BACKEND WEB ---
6  web:
7    build: .      # Costruisci l'immagine dal Dockerfile corrente
8    container_name: my_python_app
9    ports:
10     - "5000:5000" # Esposizione: Host:Container
11    volumes:
12     - ./app      # Bind Mount: Sviluppo live (codice Host -> Container)
13    environment:  # Variabili d'ambiente
14     - DB_HOST=db # Nota: 'db' è il nome del servizio sotto
15     - DB_PASS=secret
16    depends_on:   # Ordine di avvio
17     - db
18    networks:
19     - backend-net
20
21  # --- SERVIZIO 2: IL DATABASE ---
22  db:
23    image: postgres:13 # Usa un'immagine pronta da Docker Hub
24    restart: always    # Riavvia sempre se crasha
25    environment:
26     POSTGRES_PASSWORD: secret
27     POSTGRES_DB: myappdb
28    volumes:
29     - pg-data:/var/lib/postgresql/data # Volume Named (Persistenza reale)
30    networks:
31     - backend-net
32
33  # Definizione dei Volumi (Storage persistente gestito da Docker)
34  volumes:
35  pg-data:
36
37  # Definizione delle Reti
38  networks:
39  backend-net:
40    driver: bridge
41
```

6.3 Analisi delle Keywords

- **services:** L'elenco dei container da lanciare. Nel codice sopra: **web** e **db**.
- **build vs image:**
 - **build:** `.` dice "Crea l'immagine usando il Dockerfile in questa cartella".
 - **image:** `postgres` dice "Scarica l'immagine ufficiale pronta all'uso".
- **depends_on:** Controlla l'ordine di avvio. Dice "Avvia **web** solo dopo aver avviato **db**". *Attenzione: Docker attende solo che il container DB sia "running", non che il processo Postgres sia pronto ad accettare connessioni. Nel codice applicativo serve spesso una logica di "retry" o uno script "wait-for-it".*
- **networks:** Definisce il perimetro di sicurezza. I container in reti diverse non possono parlarsi.

6.4 Il Workflow Operativo

Come si usa nella vita quotidiana?

- **Avvio:** `docker compose up -d`
Legge il file, crea la rete, i volumi e avvia i container in background (Detached).
- **Stop e Pulizia:** `docker compose down`
Ferma i container e **rimuove** la rete virtuale. *Nota: Di default NON cancella i volumi (i dati del DB sono salvi). Per cancellare anche i dati, usare `-volumes`.*
- **Log:** `docker compose logs -f`
Mostra un output aggregato dei log di tutti i servizi in tempo reale (utile per debugging incrociato).
- **Ricostruzione:** `docker compose up -d -build`
Forza la ricompilazione dell'immagine se hai cambiato il Dockerfile o le dipendenze.

7 Comandi Cheat-Sheet

- `docker build -t nome:tag .` : Costruisce l'immagine.
- `docker run -d -p 80:80 -name web nginx` : Avvia in background, mappa porta 80 host su 80 container.
- `docker ps -a` : Lista container (anche fermi).
- `docker exec -it <id> bash` : Apre una shell dentro un container attivo.
- `docker logs -f <id>` : Segue i log in tempo reale.
- `docker system prune -a` : Pulisce tutto (immagini non usate, container fermi).
- `docker compose up -d -build` : Avvia l'app ricompilando se necessario.